Getting Started with Spatial SQL

Updated: Jan 9, 2025 | Kent Marten Michael Johns

Thank you for participating in the Spatial SQL Private Preview with Databricks. The following guide will show you how to enable the preview and provide examples for types of queries you can now execute. *There is also a corresponding, and fairly comprehensive, notebook titled DBR 14.2 Getting Started: Spatial SQL Preview [v1] that you should also receive as part of onboarding to the Preview.*

Enable

This enablement config is only required during the Private Preview when using Databricks Runtime 14.2+. The preview is supported in DBSQL and with Serverless compute, enabled by workspace_id by the Databricks team.

For classic compute, use the following spark configs for enablement.

SQL

```
set spark.databricks.geo.st.enabled=true;
```

Scala, Python

```
spark.conf.set("spark.databricks.geo.st.enabled", "true")
```

Disable

SQL

```
set spark.databricks.geo.st.enabled=false;
```

Scala, Python

```
spark.conf.set("spark.databricks.geo.st.enabled","false")
```

Using Geometry/Geography types

In general, Databricks spatial expressions support reading from Well-known Text (WKT), Well-known Binary (WKB), Extended Well-known Binary (EWKB) and GeoJSON. You can also convert to native geospatial types (Geography and Geometry) and use these within a query. For the preview, you cannot materialize Geography and Geometry types or return them in a resultset,, so you need to convert to WKT, WKB, EWKB, or GeoJSON prior to storing or including in final results.

The preview offers a number of overloaded signatures for ST_ functions to allow convenience when starting from WKT (String), WKB (Binary), and GeoJSON (String) columnar data as those can be passed as parameters. For example if you want to check whether a column with GeoJSON contains 2D spatial data, you can invoke `ST_NDims(<geojson_col>)` directly versus `ST_NDims(ST_GeomFromGeoJSON(<geojson_col>))`. However, in a chained query the return type standardized to the in-memory types has benefits, e.g. in the chain `ST_Buffer(ST_Envelope(<geojson_col>))`, both `ST_Buffer` and `ST_Envelope` return a GEOMETRY type, with `ST_Buffer` only accepting GEOMETRY param and `ST_Envelope` supporting overloads, so the standardization to GEOMETRY lends to efficiency since the in-memory type can be reused once generated.

When you want to work with in-memory GEOGRAPHY types, e.g. for `ST_GeogArea`, `ST_GeogLength`, and `ST_GeogPerimeter` available in the preview, you can construct using calls like `ST_GeogFromWKT`, `ST_GeogFromWKB`, and `ST_GeogFromGeoJSON`, optionally specifying the SRID for WKT and WKB variants, default is 4326 (WGS84).

Optimizing Spatial Joins with H3

Important: The preview does not automatically perform spatial indexing on GEOMETRY/GEOGRAPHY columns, and spatial joins will not be performant unless customers take explicit actions as described below.

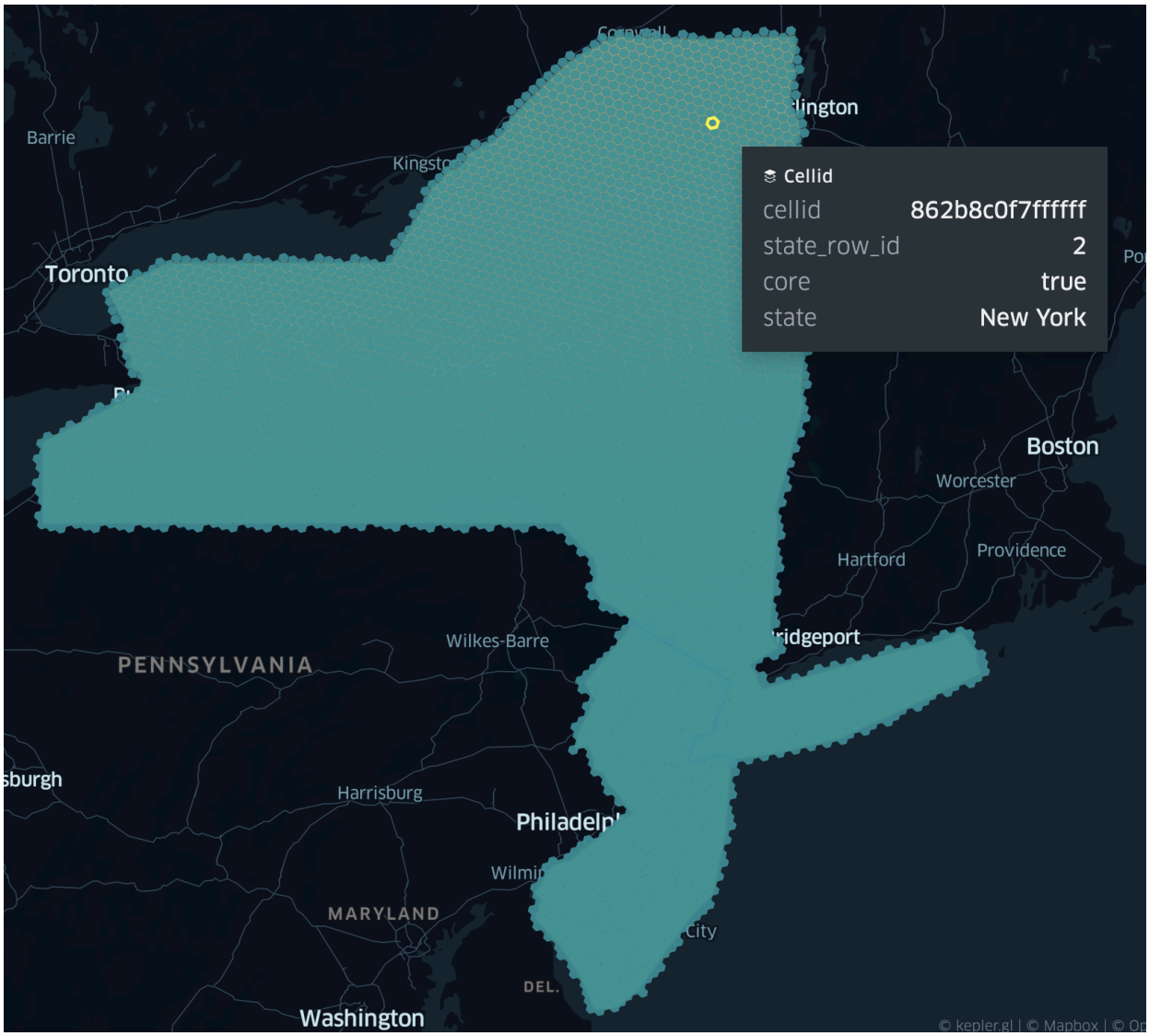
Data Engineering

The preview supports, and recommends use of, `INLINE(H3_TessellateAsWKB(<geo>,<res>)` to effectively spatially index spatial data into chips at a given h3 resolution. `INLINE` explodes the resulting array from `H3_TessellateAsWKB` and shreds, or flattens, the struct fields into three top-level columns: `cellid` containing h3 cell id, `core` to identify a chip being core vs boundary, and `chip` with the WKB. Table layout should be optimized by `cellid` using Liquid Clustering.

```
-- table will use liquid clustering
-- The use of inline allows us to cluster by 'cellid'
-- NOTICE: we do not include the full geometry 'g' here;
--         rather, keep only vector chips per cellid
CREATE OR REPLACE TABLE state_h3 CLUSTER BY (cellid) AS
(
 SELECT state_row_id, INLINE(h3_tessellateaswkb(g, 6)), * EXCEPT(state_row_id, g)
 FROM state_poly
);

-- execute liquid clustering
OPTIMIZE state_h3;
```

	state_row_id	cellid	core	1.2	chip	state
1	1	604222584191451135	true	AQMAAAABAAAABwAAAEyryLgnmlLAsXwZaYI7REBuW9j2/ZxSwAy2ks+LOkRAysw/...		New Jersey
2	1	604222540839124991	true	AQMAAAABAAAABwAAAA83vC0slFLAfLh81d7dQ0AP3St095ZSwGrM2Izr3ENAdfK...		New Jersey
3	1	604222345149677567	true	AQMAAAABAAAABwAAABbRyktLmlLAYtuBvt2QREAJaw3sK51SwJ0/jGHlj0RAIlOm5...		New Jersey
4	1	604233111693164543	false	AQMAAAABAAAABgAAAMDJ9HwMw1LAzKsfKXNsQ0BSs2ASicJSwPRu78TzbkNAb...		New Jersey
5	1	604222508626870271	true	AQMAAAABAAAABwAAAB7aJXw/uVLAjMAWw+5vREBU5x4AG7xSwFj64ujwbkRAFP...		New Jersey
6	1	604222563119267839	false	AQMAAAABAAAACAAAANpyIXWDe1LAgBYjX40HRECRt4LEm3tSwAhr/sAeB0RArFY...		New Jersey
7	1	604222324077494271	true	AQMAAAABAAAABwAAANG3RK9YglLAiXtumTpsRECERe+bNYVSwIlUar1Ha0RA3RR...		New Jersey
8	1	604222574259339263	true	AQMAAAABAAAABwAAAENFZVv3hVLAciONU7LXQ0CqP89awohSwLuZNLrB1kNA5...		New Jersey
9	1	604233090620981247	true	AQMAAAABAAAABwAAALUl3Vxdx1LAJDPNKiLtQ0CJVayvKMpSwNB85Xkk7ENAWt...		New Jersey
10	1	604222530907013119	true	AQMAAAABAAAABwAAANwdm+KGo1LA1yjSmvLqQ0DEphwzU6ZSwLjrquL76UNAS...		New Jersey

Below we show a layer with h3 cellids that cover the geometries as well as a layer with the original geometry. Not shown here, but If the chips were all unioned back together, you would have the original geometry.



Data engineering for points is straightforward.

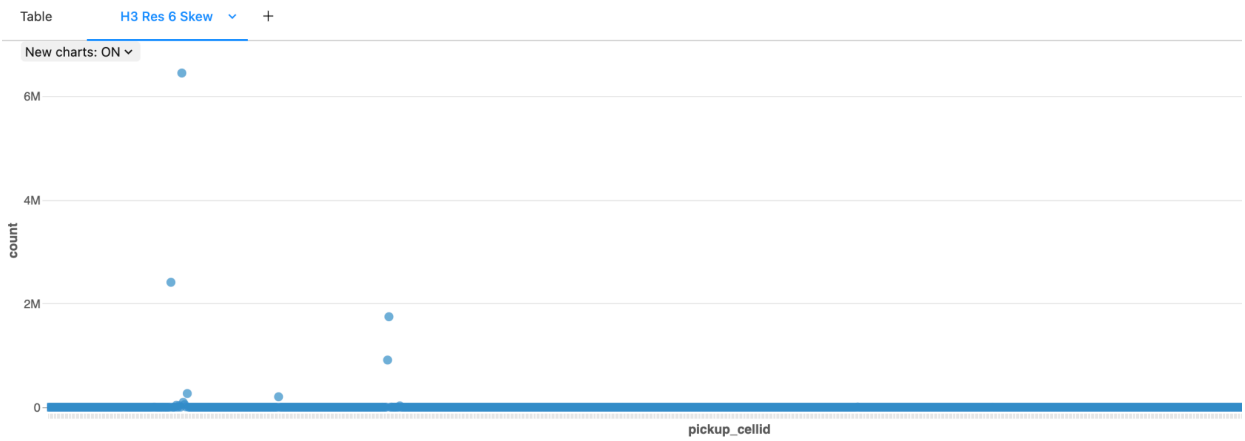
```
-- table will use liquid clustering
-- for points, we are just adding additional column(s)
CREATE OR REPLACE TABLE trip_h3 CLUSTER BY (pickup_cellid) AS
(
 SELECT h3_longlatash3(pickup_longitude, pickup_latitude, 6) AS pickup_cellid, *
 FROM trip
);

-- execute liquid clustering
OPTIMIZE trip_h3;
```

	pickup_cellid	trip_row_id	vendor_id	pickup_datetime	dropoff_datetime	passenger_count
1	604222325017018367	-7448169589944065486	CMT	2011-03-01T20:39:38.000+00:00	2011-03-01T20:52:20.000+00:00	1
2	604222325017018367	-1570006345918717334	CMT	2011-03-03T01:12:54.000+00:00	2011-03-03T01:18:06.000+00:00	1
3	604222325017018367	8089183030801806245	CMT	2011-03-01T20:42:32.000+00:00	2011-03-01T20:50:17.000+00:00	1
4	604222325017018367	7483561277756808349	CMT	2011-03-01T17:29:46.000+00:00	2011-03-01T17:35:34.000+00:00	1
5	604222325017018367	1500843736679943519	CMT	2011-03-01T14:42:34.000+00:00	2011-03-01T14:49:10.000+00:00	1
6	604222325017018367	5273529273602763647	CMT	2011-03-04T08:18:43.000+00:00	2011-03-04T08:36:24.000+00:00	1
7	604222325017018367	-42586459111083255	CMT	2011-03-01T17:05:05.000+00:00	2011-03-01T17:09:50.000+00:00	1
8	604222325017018367	4341314510927938488	CMT	2011-03-01T13:11:43.000+00:00	2011-03-01T13:15:21.000+00:00	1
9	604222325017018367	4608524376989871856	CMT	2011-03-02T06:53:50.000+00:00	2011-03-02T07:21:02.000+00:00	1
10	604222325017018367	8747532987195416115	CMT	2011-03-01T08:26:08.000+00:00	2011-03-01T08:28:38.000+00:00	1

Look closer at trips data

The average cell area for h3 resolution 6 is 36km^2, so with a quick plot of NYC, we can see that there are multiple outliers above ~1M which we would want to remain mindful of when considering join skew (especially since this is only a 1% sample).

```
select
  pickup_cellid, count(1) as count
from trip_h3
group by pickup_cellid
order by count desc
```



Spatial Joins

The preview supports, and recommends use of, additional `WHERE` clauses to first test for h3 `cellid` matches and then only invoke more intensive spatial predicates, such as `ST_Contains` or `ST_Intersects`, when a boundary chip is involved (`core = false`).

The examples below, [1] and [2], are adapted from, and extend, this notebook, which we used in a previous h3 blog series. The difference is that in the preview, these functions are now provided without requiring any external library. *A more detailed example of accomplishing data engineering steps for performant spatial joins can be found within the Getting Started Notebook that goes with this guide.*

[1]: H3-Approximate Query

```
-- Use "pure" H3 for fast and approximate comparisons
create table if not exists approx_pickup_zone as (
select
taxi_zone_h3_chip.*,
t.*
from (select /*+ SKEW('pickup_cellid') */ * from trip_h3) as t
join taxi_zone_h3_chip
on cellid == pickup_cellid
)
```

[2]: H3-Precise Query [Hybrid Pattern]

```
-- For precise (using H3 spatial indexing) add clause:
--  where <core_col> or st_contains(<geoExpr1>,<geoExpr2>)
create table if not exists precise_pickup_zone as (
select
taxi_zone_h3_chip.*,
t.*
from (select /*+ SKEW('pickup_cellid') */ * from trip_h3) as t
join taxi_zone_h3_chip
on cellid == pickup_cellid
where core or st_contains(st_geomfromwkb(chip), st_point(pickup_longitude, pickup_latitude))
)
```

H3 "Hybrid" Spatial Queries

For tables prepared with h3 based data engineering, as recommended, non-point geometries will have been exploded into row-per-`cellid`. Assuming the presence of a geo id (in this case 'state_row_id'), you can drop the h3 affiliated fields to coerce a GEOMETRY/GEOGRAPHY centric response. Further, `ST_Union_Agg` can be used to aggregate h3 chips into a single GEOMETRY/GEOGRAPHY, e.g. through use of `Group By` on a geo id.

[1]: Geometry-Centric Results via Chip Union

```
-- we can union h3 chips back together, e.g. ones involved in the results
select state_row_id, state, st_astext(st_union_agg(chip)) as chip_union from
 (select distinct state_row_id, state, chip from pickup_state_h3)
 group by state_row_id, state
 order by state_row_id
```

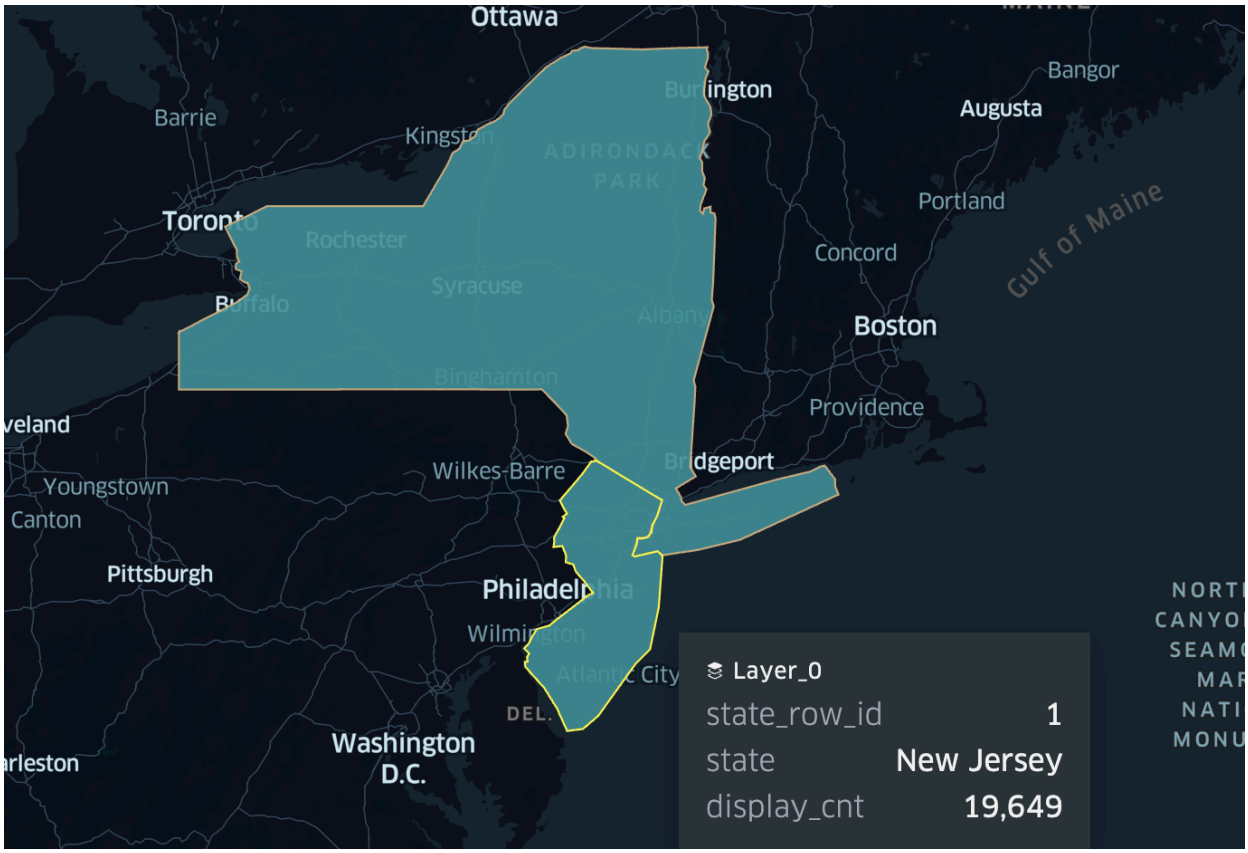
	state_row_id	state	chip_union
1	1	New Jersey	MULTIPOLYGON(((-74.3410874289631 41.1964348476457,-74.2713661277...
2	2	New York	MULTIPOLYGON(((-73.4310103477268 40.5425754588093,-73.4628 40.536...

[2]: Geometry-Centric Aggregated Results

```
-- we can join back the original geometry
-- most suitable when performing aggregates
SELECT
   t.*,
   state_poly.g
 FROM (
   SELECT
     state_row_id, state, count(1) as pickup_cnt,
     format_number(count(1),0) as display_cnt
     FROM pickup_state_h3
   GROUP BY state_row_id, state
 ) as t
 join state_poly
 on t.state_row_id == state_poly.state_row_id
 ORDER BY state_row_id
```

	state_row_id	state	pickup_cnt	display_cnt	g
1	1	New Jersey	19649	19,649	POLYGON((-74.6950 41.3572,-74.6559 41.3394,-73.8940 40.9934,-73.9586 ...
2	2	New York	12232936	12,232,936	MULTIPOLYGON (((-79.7621 42.5146, -79.7621 42.5143, -79.7624 42.5142, -...

Here is the aggregate pickup counts from the point-in-polygon join per state.



[3]: Data-Centric Results

```
-- we can easily filter out the h3 related fields after a spatial join
select * except(cellid, core, chip, pickup_cellid)
from pickup_state_h3
```

	state_row_id	state	trip_row_id	vendor_id	pickup_datetime
1	2	New York	-2585005784246705694	2	2015-01-29T22:53:57.000+00:00
2	2	New York	-839845815702497166	2	2015-01-30T22:46:37.000+00:00
3	2	New York	-5012791320556840754	2	2015-01-10T10:51:51.000+00:00
4	2	New York	-805528271827049483	1	2015-01-03T12:02:46.000+00:00
5	2	New York	-6340835473749379143	2	2015-01-19T12:57:33.000+00:00
6	2	New York	-1762290991729138674	2	2015-01-18T16:55:28.000+00:00
7	2	New York	-1145417812997891684	2	2015-01-23T23:45:39.000+00:00
8	2	New York	-98326512962084350	1	2015-01-18T14:03:18.000+00:00
9	2	New York	1932556285035350970	1	2015-01-21T21:33:42.000+00:00
10	2	New York	8165977419349107944	1	2015-01-11T23:43:21.000+00:00

FAQ

Also, see [Spatial SQL - Private Preview - FAQ]

[1] What should I do to fix a query that throws [ST_UNSUPPORTED_RETURN_TYPE] error?

There is additional information in the exception stating something like the following:

> *SparkUnsupportedOperationException: [ST_UNSUPPORTED_RETURN_TYPE] The GEOGRAPHY and GEOMETRY data types cannot be returned in queries. Use one of the following SQL expressions to convert them to standard interchange formats: "st_asbinary", "st_astext", or "st_asgeojson". SQLSTATE: 0A000.*

You can correct a query such as SELECT `st_buffer(<geoExpr>)` by converting the return to WKT, WKB, or GeoJSON, e.g. SELECT `st_astext(st_buffer(<geoExpr>))` for WKT.

[2] How do I initially get my data into the Lakehouse to use spatial sql preview?

Please reach out to your account team for guidance on getting data into the Databricks Lakehouse. It may involve using our built-in readers, available geospatial readers, such as from one of our DBLabs projects or various 3rd party libraries. The optimal format to target is Delta Lake which Databricks specially leverages in both DBR Clusters and DBSQL. There are some pretty straightforward patterns to process data from various formats into Delta Lake, so please let us know if you need assistance.

ST_ Functions

	Input type(s)	Output type
h3_tessellateaswkb	(STRING, INT[, BOOLEAN])/(GEOGRAPHY, INT[, BOOLEAN])/(BINARY, INT[, BOOLEAN])	STRUCT{BIGINT, BOOLEAN, BINARY}

Description	Tessellate input vector data using h3 global grid indexing. The expression takes as input two required and one optional argument. More on other productized h3 functions here. • A geography object (either a GEOGRAPHY value, or a STRING/BINARY value representing a geography in GeoJSON, WKB, or WKT format). • A resolution value (value between 0 and 15, inclusive). • [Optional] A boolean value indicating whether the WKB description of core H3 cells should be present in the output, or replaced by NULL values (see below). The function returns an array of named structs. The array represents a tessellation of the input geography using H3 cells at the specified resolution. The tessellation is essentially a decomposition of the geography using a minimal covering set of H3 cells. The structs have three fields: "cellid", "core", "chip". The "cellid" is an H3 cell at the specified resolution that is a member of the minimal covering set of H3 cells of the geography at the specified resolution. The "core" field is a boolean value that indicates whether the H3 cell of the struct is fully contained in the geography (for core cells the intersection of the geography with the cell's polygon is the cell's polygon). The "chip" field is a BINARY value, and corresponds to the intersection of the input geography with the cell's polygon, represented in WKB format. If the case where the cell is fully contained inside the geography and the third optional argument is set to false, the "chip" field value would be NULL instead.
Notes	`h3_tessellateaswkb` allows vector chipping of spatial data to establish a spatial index, which is essential to achieve scaled performance. Without H3 indexing, you as a user will have a poor experience even before hitting large scales, see discussion from previous blog on the impact of not having any index. v1 of the preview has no implicit / managed indexing of your spatial data. Read on below to understand more about how to data engineer your base geometry tables to establish spatial indexing. • The expression disregards any Z or M coordinates that the input geography may have. • The WKB returned as the third field of the structs is a 2D geography. • The expression does not support geometry collections as input. An error is returned if the input is a geometry collection. • The expression will throw an error if the resolution is outside the valid range. • The expression will throw an error if the input is a STRING/BINARY value that is an invalid GeoJSON, WKB, or WKT representation of a geography. The input geography is expected to have coordinates in the WGS84 coordinate reference system. It is best practice to use INLINE to shred the table, effectively exploding the array of structs and making the 3 fields within the struct ("cellid", "core", "chip") top level columns.

Example(s)	`-- THREE OPTIONS [#3 RECOMMENDED]`
	`-- These are from the Getting Started Notebook`
	
	`-- [1] tessellating at h3 resolution 6`
	`-- returns an array per geom, so doesn't explode or perform table shredding`
	`-- !!! This is not recommended for scaled performance !!!`
	`select`
	`  h3_tessellateaswkb(g, 6) as tessellate, *`
	`from state_poly`

	tessellate	state_row_id	state	g
0	[{'cellid': 604248381912514559, 'core': True, ...	2	New York	MULTIPOLYGON (((-79.7621 42.5146, -79.7621 42....

```
-- [2] tessellating at h3 resolution 6 + `explode`
-- returns a row per h3 cell per geom, but doesn't perform struct shredding
-- !!! This is not recommended for optimized table ordering, e.g. with liquid clustering !!!
select
  explode(h3_tessellateaswkb(g, 6)) as tessellate,
  * except (g)
from state_poly
```

	col	state_row_id	state
0	{'cellid': 604248381912514559, 'core': True, '...	2	New York
1	{'cellid': 604248585252372479, 'core': True, '...	2	New York
2	{'cellid': 604248178572656639, 'core': True, '...	2	New York
3	{'cellid': 604248335875833855, 'core': True, '...	2	New York
4	{'cellid': 604249509878300671, 'core': True, '...	2	New York
5	{'cellid': 604232632670093311, 'core': True, '...	2	New York
6	{'cellid': 604222622980374527, 'core': False, ...	2	New York
7	{'cellid': 604248243802472447, 'core': True, '...	2	New York
8	{'cellid': 604248447142330367, 'core': True, '...	2	New York
9	{'cellid': 604222373603835903, 'core': True, '...	2	New York

```
-- [3] tessellating at h3 resolution 6 + `inline`
--     returns a row per h3 cell per geom with struct shredding, meaning:
--     'cellid', 'core', and 'chip' become top-level fields
--     !!! This is recommended for scaled performance and optimized table ordering !!!
select inline(h3_tessellateaswkb(g, 6)), * except (g) from state_poly where state = "New York"
limit 10
```

	cellid	core	chip	state_row_id	state
0	604248381912514559	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
1	604248585252372479	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
2	604248178572656639	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
3	604248335875833855	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
4	604249509878300671	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
5	604232632670093311	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
6	604222622980374527	False	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x05\x00...	2	New York
7	604248243802472447	True	b'\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
8	604248447142330367	True	b"\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York
9	604222373603835903	True	b"\x01\x03\x00\x00\x00\x01\x00\x00\x00\x07\x00...	2	New York

	Input type(s)	Output type
st_area	GEOMETRY / GEOGRAPHY / STRING / BINARY	DOUBLE
st_geogarea	STRING / BINARY	DOUBLE
Description	Returns the area of a polygonal geometry. Accepts STRING or BINARY inputs representing a geography/geometry in GeoJSON, WKB, or WKT format.	
Notes	In the case of `ST_Area` the input value is understood as a geometry. • For points, linestrings, multipoints, and multilinestrings we always return 0. • For polygons we return the 2D Cartesian area of the polygon. • For multipolygons we return the sum of the areas of the polygons in the multipolygon. • For geometry collections we return the sum of the areas of the elements in the collection. • The units of the result are those of the coordinates of the geometry. In the case of `ST_GeogArea` the input value is understood as a geography. • We expect the input coordinates to be in degrees, and the underlying coordinate system is assumed to be WGS84. • For points, linestrings, multipoints, and multilinestrings we always return 0. • For polygons we return the 2D geodesic area of the polygon. • For multipolygons we return the sum of the areas of the polygons in the multipolygon. • For geometry collections we return the sum of the areas of the elements in the collection. • The units of the result are squared meters. For both expressions we return an error if the input geography/geometry representation is not a valid GeoJSON, WKB, or WKT.	

Example(s)	`-- NOTICE: difference between `st_geogarea` [meters]`
	`--  and `st_area` [units: e.g. degrees]`
	`with samp_wkt as (`
	`  select`
	`  1 as row_id,`
	`  'POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276,`
	`-115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581`
	`32.5700043540444))' as g`
	`)`
	`select`
	`  st_geogarea(g) as dbx_area_meters,`
	`  st_area(g) as dbx_area_units, *`
	`from samp_wkt`	

	1.2 dbx_area_meters	1.2 dbx_area_units	row_id	g
1	67.75894184472462	6.502873069828407e-9	1	POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32...

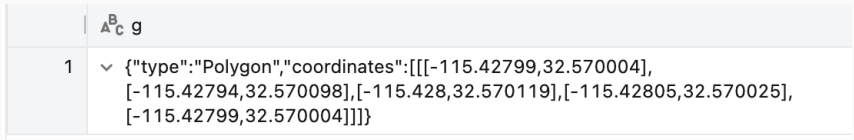
	Input type(s)	Output type
st_asbinary / st_aswkb	(GEOMETRY[, STRING]) / (GEOGRAPHY[, STRING])	BINARY
Description	Returns Well-Known Binary (WKB) representation of the geometry/geography input. The optional second parameter is to specify endian encoding, either little-endian ('NDR') or big-endian ('XDR').	
Notes	Input must be a geometry or geography type.	

Example(s)	`-- we are initially converting from WKT to geometry type for convenience`
	`select st_asbinary(st_geomfromtext(`
	`  'POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276,`	

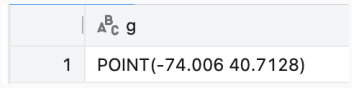
	-115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))')) as g 1.2 g 1 AQMAAAABAAAABQAAAOKHvjFk21zAYpkV5/VlQECPakZyY9tcwLiGGPv4SEBAk82...

	Input type(s)	Output type
st_asewkb	(GEOMETRY[, STRING]) / (GEOGRAPHY[, STRING])	BINARY
Description	Returns Extended Well-Known Binary (EWKB) representation of the geometry/geography input. The optional second parameter is to specify endian encoding, either little-endian ('NDR') or big-endian ('XDR'). More on EWKB here.	
Notes	Input must be a geometry or geography type.	
Example(s)	-- we are initially converting from WKT to geometry type for convenience -- showing optional big-endian param select st_asewkb(st_geomfromtext('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))'), 'XDR') as g 1.2 g 1 AAAAAAMAAAABAAAABcBc22QxvofiQEBI9ecVmWLAXNtjckZqj0BASPj7Gla4wFzbZ...	

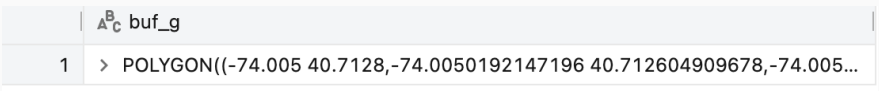
	Input type(s)	Output type
st_asgeojson	(GEOMETRY[, INT]) / (GEOGRAPHY[, INT])	STRING
Description	Returns GeoJSON representation of the geometry/geography input. The optional second parameter is to specify precision.	
Notes	Input must be a geometry or geography type.	
Example(s)	-- we are initially converting from WKT to geometry type for convenience -- [1] without optional precision select st_asgeojson(st_geomfromtext('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))')) as g g 1 {"type":"Polygon","coordinates":[[[-115.427990375581,32.5700043540444], [-115.427944725767,32.5700982923276],[-115.428003926578,32.5701189200886], [-115.428049576338,32.5700249817816],[-115.427990375581,32.5700043540444]]]} -- [2] with optional precision=8 select st_asgeojson(st_geomfromtext('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))'), 8) as g	

	Input type(s)	Output type
st_astext / st_aswkt	(GEOMETRY[, INT]) / (GEOGRAPHY[, INT])	STRING
Description	Returns Well-Known Text (WKT) representation of the geometry/geography input. The optional second parameter is to specify precision.	
Notes	Input must be a geometry or geography type.	

Example(s)	`-- we are initially converting from WKT to geometry type for convenience`
	`select st_astext(st_geomfromtext('POINT(-74.0060 40.7128)')) as g`
	 	

	Input type(s)	Output type
st_buffer	GEOMETRY, DOUBLE	GEOMETRY
Description	Computes a POLYGON or MULTIPOLYGON that represents all points whose distance from a geometry is less than or equal to a given distance.The expression takes as input a geometry and a radius, and returns the buffered geometry using the input radius.	
Notes	The buffer of a geometry is the Minkowski sum or difference (depending or whether the radius is positive or negative, respectively) of the geometry with a disk of the given (absolute) radius. Regarding the SRID value of the result: • If the input value is of type GEOMETRY, the SRID value of the output GEOMETRY value is equal to that of the input value. • If the input value is of type BINARY, the SRID value of the output GEOMETRY value is 0. • If the input value is of type STRING and represents a geometry in WKT format, the SRID value of the output GEOMETRY value is 0. • If the input value is of type STRING and represents a geometry in GeoJSON format, the SRID value of the output GEOMETRY value is 4326. The result is always a 2D polygon or multipolygon (in other words, we drop the Z and M coordinates from the input geometry).	

Example(s)	`-- radius 0.001 specified (same units as input)`
	`select st_astext(st_buffer(st_geomfromtext('POINT(-74.0060 40.7128)'), 0.001)) as buf_g`
	 	

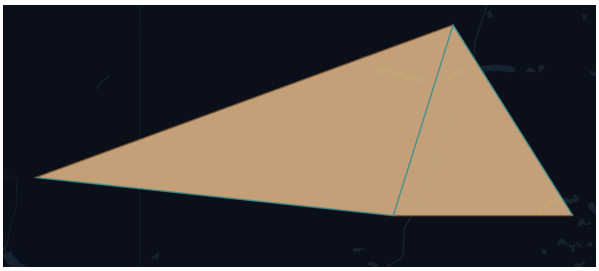
	Input type(s)	Output type
st_centroid	GEOMETRY	GEOMETRY

Description	Expression takes as input a geometry object and returns the centroid of that geometry as a 2D point.
Notes	A few details about the behavior: • If the input geometry is empty, the 2D empty point is returned. • If the input geometry consists of points only, the centroid is the average of the X and Y coordinates of the points. • If the input geometry contains linear segments (but no areal geometries), the centroid is the weighted average of the midpoints of the linear segments, where the weights are the lengths of the segments. • If the input geometry contains polygons, the centroid is the weighted average of the centroids of the polygons, where the weights are the areas of the polygons. • In case of mixed topological dimension components, the centroid computation is based on the components of highest topological dimension. • If the input is a BINARY value, it is expected to be the WKB description of a geometry. • If the input is a STRING value, it is expected to be the GeoJSON or WKT description of a geometry. • If the input is GeoJSON, the SRID of the resulting geometry is 4326. • If the input is WKB or WKT, the SRID of the resulting geometry is 0. • If the input is a GEOMETRY value, the SRID value of the output geometry is the same as that of the input value.
Example(s)	`select st_astext(st_centroid(g)) as c, g from samp_line` c: POINT(-100.338283744525 50.3351256593501) g: LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)

	Input type(s)	Output type
st_contains	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	BOOLEAN
st_intersects	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	BOOLEAN
Description	Two common spatial predicate expressions: • `ST_Contains`: takes as input two geometries and returns true iff the first geometry contains the second. • `ST_Intersects`: takes as input two geometries and returns true iff the two geometries intersect.	
Notes	Spatial predicates are more compute intensive. *For performance purposes, we strongly urge customers to spatially index their data using productized functions h3_lonlatash3 and h3_tessellateaswkb.* Some requirements or details on the behavior: • Both expressions take either two GEOMETRY values as input or any combination of STRING and BINARY inputs. In the latter case the inputs are expected to be standard geospatial representation of geometries (GeoJSON, WKB, or WKB). • Both expressions require that both inputs are not geometry collections, otherwise an error is returned. • If the two inputs are GEOMETRY values, they are expected to have the same SRID value (otherwise an error is returned).	

	The notion of containment or intersection that is implemented follows the DE-9IM matrix semantics.

Example(s)	`-- assign taxi pickups a state using st_contains`
	`-- for simplicity, just showing the ~12.2M result count`
	`-- h3 is the spatial indexing used to drive performance`
	`select`
	`format_number(count(1),0) as count`
	`from`
	`(select /*+ SKEW('pickup_cellid') */ * from trip_h3) as t`
	`join state_h3`
	`on cellid == pickup_cellid`
	`where`
	`core or`
	`st_contains(st_geomfromwkb(chip), st_point(pickup_longitude, pickup_latitude))`
	
	| | count |
	| --- | --- |
	| 1 | 12,252,585 |

	Input type(s)	Output type
st_convexhull	GEOMETRY / STRING / BINARY	GEOMETRY
Description	The expression takes as input a geometry and returns the convex hull of the input geometry as a GEOMETRY value.	
Notes	The expression has overloads for BINARY, GEOMETRY, and STRING inputs. In the case of BINARY and STRING inputs we expect the values to be valid representations of a geometry in one of the standard geospatial formats GeoJSON (STRING), WKB (BINARY), and WKT (STRING).	
Example(s)	`select *, st_astext(st_convexhull(g)) as cg from samp_line` | | row_id | g | cg | | --- | --- | --- | --- | | 1 | 1 | LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05) | POLYGON((-100.3 50.05,-100.5 50.4,-100.25 50.5,-100.25 50.35,-100.3 50.05)) | Using map_render utility function in Getting Started notebook (blue is 'g' and tan is 'cg'): 

	Input type(s)	Output type
st_difference	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	GEOMETRY
st_intersection	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY,	GEOMETRY

	STRING) / (BINARY, BINARY)	
st_union	(GEOMETRY, GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	GEOMETRY
Description	Three geospatial expressions related to boolean set operations: • `ST_Difference`: takes as input two geometries and returns their point-set difference as a GEOMETRY value. • `ST_Intersection`: takes as input two geometries and returns their point-set intersection as a GEOMETRY value. • `ST_Union`: takes as input two geometries and returns their point-set union as a GEOMETRY value.	
Notes	• All three expressions take either two GEOMETRY values as input or any combination of STRING and BINARY inputs. In the latter case the inputs are expected to be standard geospatial representation of geometries (GeoJSON, WKB, or WKB). • All three expressions require that both inputs are not geometry collections, otherwise an error is returned. • If the two inputs are GEOMETRY values, they are expected to have the same SRID value (otherwise an error is returned). In this case the SRID value of the returned GEOMETRY value is equal to the common SRID value of the inputs. • If the two inputs are STRING/BINARY values then the SRID value of the output GEOMETRY is either 4326 (if both inputs are in GeoJSON format), or 0 (all other cases).	

Example(s)	`-- difference`
	`select`
	` st_astext(st_difference('LINESTRING(0 0,0 2)', 'LINESTRING(0 1,0 2)')) as overlap_lines,`
	` st_astext(st_difference('LINESTRING(0 0,0 2)', 'LINESTRING(1 2,1 3)')) as non_overlap_lines`
	
	| | overlap_lines | non_overlap_lines |
	| 1 | LINESTRING(0 0,0 1) | LINESTRING(0 0,0 2) |	

	`-- intersection`
	`select`
	` st_astext(st_intersection('LINESTRING(0 0,0 2)', 'LINESTRING(0 1,0 2)')) as overlap_lines,`
	` st_astext(st_intersection('LINESTRING(0 0,0 2)', 'LINESTRING(2 2,2 3)')) as non_overlap_lines`
	
	| | overlap_lines | non_overlap_lines |
	| 1 | LINESTRING(0 1,0 2) | LINESTRING EMPTY |	

	`-- union`
	`select`
	` st_astext(st_union('LINESTRING(0 0,0 2)', 'LINESTRING(0 1,0 2)')) as overlap_lines,`
	` st_astext(st_union('LINESTRING(0 0,0 2)', 'LINESTRING(2 2,2 3)')) as non_overlap_lines`
	
	| | overlap_lines | non_overlap_lines |
	| 1 | MULTILINESTRING((0 0,0 1),(0 1,0 2)) | MULTILINESTRING((0 0,0 2),(2 2,2 3)) |	

	Input type(s)	Output type
st_distance	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	DOUBLE

st_distancesphere	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	DOUBLE
st_distancespheroid	(GEOMETRY,GEOMETRY) / (STRING, STRING) / (STRING, BINARY) / (BINARY, STRING) / (BINARY, BINARY)	DOUBLE
Description	2D Euclidean • `ST_Distance`: takes as input two geometries, or two standard representations of geometries (GeoJSON, WKB, or WKB), and returns the 2D Euclidean distance of the two geometries. The units of the returned distance are those of the input geometries. WGS84 Ellipsoid • `ST_DistanceSphere`: takes as input two point geometries and returns their spherical distance in meters, measured on a sphere whose radius is the _mean radius of the WGS84 ellipsoid_. The expression can also take STRING/BINARY values as input, representing geometries in GeoJSON, WKB, or WKT format. • `ST_DistanceSpheroid`: takes as input two point geometries and returns their geodesic distance in meters, _measured on the WGS84 ellipsoid_. The expression can also take STRING/BINARY values as input, representing geometries in GeoJSON, WKB, or WKT format.	
Notes	For `ST_Distance`: • If at least one of the geometries is empty, NULL is returned. • If the input geospatial representations are not valid, a parse error is returned. • If the two inputs are GEOMETRY values with different SRIDs, an error is returned. For both `ST_DistanceSphere` and `ST_DistanceSpheroid`: • The geometries are expected to have coordinates in degrees. • The expression returns NULL if at least one of the two input point geometries is empty. • The expression returns an error if at least one of the two input geometries is not a point. • In the case both inputs are GEOMETRY values, they are expected to have the same SRID value. Otherwise an error is returned.	

Example(s)	`with samp_pnts as (`
	`  select 'POINT(-74.0060 40.7128)' as nyc, 'POINT(-77.0257 38.9005)' as dc`
	`)`
	`-- notice ~3.5 degrees vs 327K meters`
	`-- sphere uses the mean radius of the WGS84 ellipsoid`
	`-- spheroid uses the full WGS84 ellipsoid datum`
	`select`
	`  format_number(st_distance(nyc,dc), 8) as dist_degrees,`
	`  format_number(st_distancesphere(nyc,dc), 4) as dist_sphere_meters,`
	`  format_number(st_distancespheroid(nyc,dc), 4) as dist_spheroid_meters`
	`from samp_pnts`	

	dist_degrees	dist_sphere_meters	dist_spheroid_meters
1	3.52179207	327,295.8765	327,620.3371

	Input type(s)	Output type

st_envelope	GEOMETRY / STRING / BINARY	GEOMETRY
Description	Expression takes as input a geometry and returns a 2D geometry representing the axis-aligned 2D minimum bounding box of the input geometry (if one exists).	
Notes	In more detail: • If the input geometry is empty, the algorithm returns a copy of the input geometry. • If the bounding box of the input geometry degenerates to a point, the algorithm returns that point. • If the bounding box degenerates to an axis-aligned segment, the algorithm returns a linestring with 2 points corresponding to the two endpoints of the segment. The points of the linestring are lexicographically ordered. • In all other cases the algorithm returns a polygon with a single ring and 4 distinct vertices (the number of points is 5 since the ring needs to be closed). The ring/polygon is clockwise-oriented and its first (and last) point is the lexicographically minimum point of the axis-aligned minimum bounding box. In all cases, the SRID of the returned geometry is equal to the SRID of the input geometry. Although it should be clear from the description above, for non-empty input, the Envelope algorithm: • Returns a 2D geometry. • The Z and M coordinates of the input geometry are ignored (if they exist).	
Example(s)	`select st_astext(st_envelope('LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)')) as env_line, st_astext(st_envelope('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))')) as env_poly` env_line: POLYGON((-100.5 50.05,-100.5 50.5,-100.25 50.5,-100.25 50.05,-100.5 50.05)) env_poly: POLYGON((-115.428049576338 32.5700043540444,-115.428049576338 32...	

	Input type(s)	Output type
st_geogfromgeojson	STRING	GEOGRAPHY
st_geogfromtext / st_geogfromwkt	STRING	GEOGRAPHY
st_geogfromwkb	BINARY	GEOGRAPHY
Description	• `ST_GeogFromGeoJSON`: reads a geography object from a GeoJSON string. • `ST_GeogFromText` / `ST_GeogFromWKT`: reads a geography object from a WKT string. • `ST_GeogFromWKB`: reads a geography object from a WKB binary.	
Notes		
Example(s)	<no example provided>	

	Input type(s)	Output type
st_geometrytype	GEOMETRY / GEOGRAPHY / STRING / BINARY	STRING
Description	The expression takes as input a GEOGRAPHY or GEOMETRY value and returns its type as a string.	

Notes	
Example(s)	`select st_geometrytype('POINT(-77.0257 38.9005)') as pnt, st_geometrytype('LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)') as line, st_geometrytype('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))') as poly`

	pnt	line	poly
1	ST_Point	ST_LineString	ST_Polygon

	Input type(s)	Output type
st_geomfromgeojson	STRING	GEOMETRY
st_geomfromtext / st_geomfromwkt	(STRING[, INT])	GEOMETRY
st_geomfromwkb	(BINARY[, INT])	GEOMETRY
st_geomfromewkb	(BINARY[, INT])	GEOMETRY
Description	• ST_GeomFromGeoJSON: reads a geometry object from a GeoJSON string. • ST_GeomFromText / ST_GeomFromWKT: reads a geometry object from a WKT string. • ST_GeomFromWKB: reads a geometry object from a WKB binary. • ST_GeomFromEWKB: reads a geometry object from a EWKB binary. The optional integer parameter, when present, is for SRID.	
Notes	GeoJSON specifies that SRID must be 4326, so SRID is not provided to ST_GeomFromGeoJSON.	
Example(s)	<no example provided>	

	Input type(s)	Output type
st_isvalid	GEOMETRY / STRING / BINARY	BOOLEAN
Description	Takes as input a geometry object and returns true iff the geometry is valid according to the OGC Simple Feature Access specification.	
Notes	A few details about the behavior: • If the input is a BINARY value, it is expected to be the WKB description of a geometry. • If the input is a STRING value, it is expected to be the GeoJSON or WKT description of a geometry. • If the input is STRING or BINARY and the description of the geometry is invalid the function returns false (does not throw a parse error).	
Example(s)	`select st_isvalid(null) as null_test, st_isvalid('POLYGON EMPTY') as empty_poly, st_isvalid('POLYGON((0 0, 10 0, 10 5, 6 -2, 0 0))') as self_intersect_poly, st_isvalid('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))') as poly`	

		null_test	empty_poly	self_intersect_poly	poly
	1	null	true	false	true

	Input type(s)	Output type
st_length	GEOMETRY / GEOGRAPHY / STRING / BINARY	DOUBLE
st_geoglength	STRING / BINARY	DOUBLE
Description	• ST_Length: the input value is understood as a geometry. For points, polygons, multipoints, and multipolygons we always return 0. For linestrings we return the sum of the 2D Cartesian lengths of the segments in the linestring. For multilinestrings we return the sum of the lengths of the linestrings in the multilinestring. For geometry collections we return the sum of the lengths of the elements in the collection. The units of the result are those of the coordinates of the geometry. • ST_GeogLength: the input value is understood as a geography. We expect the input coordinates to be in degrees, and the underlying coordinate system is assumed to be WGS84. For points, polygons, multipoints, and multipolygons we always return 0. For linestrings we return the sum of the 2D geodesic lengths of the segments in the linestring. For multilinestrings we return the sum of the lengths of the linestrings in the multilinestring. For geometry collections we return the sum of the lengths of the elements in the collection. The units of the result are meters.	
Notes	They take as input a STRING or BINARY value representing a geography/geometry in GeoJSON, WKB, or WKT format. More specifically, if the input is a BINARY value we treat it as the WKB representation, whereas if the input is a STRING value we detect whether it is GeoJSON or WKT.	
Example(s)	`-- NOTICE: difference between `st_geoglength` [meters] and `st_length` [units: e.g. degrees]` `select st_geoglength(g) as dbx_length_meters, st_length(g) as dbx_length_units, * from (select 'LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)' as g)`	

	1.2 dbx_length_meters	1.2 dbx_length_units	g
1	73146.52860234727	0.8283473425512791	LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)

	Input type(s)	Output type
st_makeline	ARRAY{GEOMETRY}	GEOMETRY
Description	Expression takes as input an array of geometries, which are expected to be points, linestrings, or multipoints. It returns a linestring geometry whose points are the non-empty points of the geometries in the input array of geometries. The order of the points is preserved in the output linestring.	
Notes	Errors: • If any of the input geometries is not a point, linestring, or multipoint an error is returned. • If the input geometries do not have the same SRID value an error is returned. Properties of the output linestring: • The SRID value of the output linestring is the common SRID value of the input geometries. • The dimension of the returned linestring is the maximum common dimension of the input geometries. • Any NULL values in the input array are ignored.	

	Some edge cases: • If the input array is empty, the 2D empty linestring is returned. The SRID of the returned linestring is 0 in this case. • If all input geometries are empty, the 2D empty linestring is returned. • If the total number of non-empty points across all input geometries is one, we return a linestring with two points, both of which are equal to the unique non-empty point in the input.

Example(s)	`-- each point from the original linestring`
	`select st_astext(st_makeline(array(`
	` st_geomfromtext('POINT(-100.25 50.5)'),`
	` st_geomfromtext('POINT(-100.50 50.40)'),`
	` st_geomfromtext('POINT(-100.25 50.35)'),`
	` st_geomfromtext('POINT(-100.30 50.05)')`
	`))) as line`
	
	1 LINESTRING(-100.25 50.5,-100.5 50.4,-100.25 50.35,-100.3 50.05)

	Input type(s)	Output type
st_makepolygon	GEOMETRY[, ARRAY{GEOMETRY}]	GEOMETRY
Description	Expression takes as input a linestring geometry and an optional array of linestring geometries. It uses these linestrings (which are expected to be closed) to construct a polygon. The outer ring of the polygon is formed from the linestring that is passed as the first argument, whereas any inner rings of the polygon are formed from the linestrings in the second argument.	
Notes	Conditions on the input values: • If any of the input geometries is not linestring, an error is returned. • If the input geometries do not have the same SRID value an error is returned. • If the outer boundary is an empty linestring, the array of inner boundaries is expected to be an empty array. Otherwise, an error is returned. Properties of the output polygon: • The SRID value of the output polygon is the common SRID value of the input geometries. • The dimension of the returned polygon is the maximum common dimension of the input linestrings. • Any NULL values in the inner boundaries array are ignored.	

Example(s)	`-- add ', -100.25 50.5' to the original linestring`
	`-- to close it as a valid polygon`
	`select st_astext(st_makepolygon(st_geomfromtext('LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05, -100.25 50.5)'))) as poly`
	
	1 POLYGON((-100.25 50.5,-100.5 50.4,-100.25 50.35,-100.3 50.05,-100.25 50.5))	

	Input type(s)	Output type
st_ndims	GEOMETRY / GEOGRAPHY / STRING / BINARY	INT

Description	Returns the number of dimensions (number of coordinates) of the points in a geography/geometry.
Notes	Takes as input a STRING or BINARY value representing a geography/geometry in GeoJSON, WKB, or WKT format. More specifically, if the input is a BINARY value we treat it as the WKB representation, whereas if the input is a STRING value we detect whether it is GeoJSON or WKT.

Example(s)	`select`
	` st_ndims(null) as nd_null,`
	` st_ndims('POINT(-77.0257 38.9005)') as nd_pnt,`
	` st_ndims('LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)') as nd_line,`
	` st_ndims('POLYGON EMPTY') as nd_poly_empty,`
	` st_ndims('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))') as nd_poly`

	nd_null	nd_pnt	nd_line	nd_poly_empty	nd_poly
1	null	2	2	2	2

	Input type(s)	Output type
st_npoints	GEOMETRY / GEOGRAPHY / STRING / BINARY	INT
st_isempty	GEOMETRY / GEOGRAPHY / STRING / BINARY	BOOLEAN
Description	• `ST_NPoints`: Returns the number of non-empty points in a geography/geometry. • `ST_IsEmpty`: Returns true if the input geography/geometry does not contain any non-empty points (it is basically equivalent to ST_NPoints(geo) = 0.	
Notes	These expressions take as input a STRING or BINARY value representing a geography/geometry in GeoJSON, WKB, or WKT format. More specifically, if the input is a BINARY value we treat it as the WKB representation, whereas if the input is a STRING value we detect whether it is GeoJSON or WKT.	

Example(s)	`select`
	` st_isempty(null) as empty_null,`
	` st_isempty('POINT EMPTY') as empty_pnt,`
	` st_isempty('POINT(-77.0257 38.9005)') as pnt`	

	empty_null	empty_pnt	pnt
1	null	true	false

```
select
  st_npoints(null) as np_null,
  st_npoints('POINT(-77.0257 38.9005)') as pnt,
  st_npoints('POLYGON EMPTY') as empty_poly,
  st_npoints('POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))') as poly
```

	np_null	pnt	empty_poly	poly
1	null	1	0	5

	Input type(s)	Output type
st_perimeter	GEOMETRY / GEOGRAPHY / STRING / BINARY	DOUBLE

st_geogperimeter	STRING / BINARY	DOUBLE
Description	`ST_Perimeter`: the input value is understood as a geometry. For points, linestrings, multipoints, and multilinestrings we always return 0. For polygons we return the sum of the 2D Cartesian lengths of the segments in the polygon. For multipolygons we return the sum of the perimeters of the polygons in the multipolygon. For geometry collections we return the sum of the perimeters of the elements in the collection. The units of the result are those of the coordinates of the geometry. `ST_GeogPerimeter`: the input value is understood as a geography. We expect the input coordinates to be in degrees, and the underlying coordinate system is assumed to be WGS84. For points, linestrings, multipoints, and multilinestrings we always return 0. For polygons we return the sum of the 2D geodesic lengths of the segments in the polygon. For multilpolygons we return the sum of the perimeters of the polygons in the multipolygon. For geometry collections we return the sum of the perimeters of the elements in the collection. The units of the result are meters.	
Notes	These expressions take as input a STRING or BINARY value representing a geography/geometry in GeoJSON, WKB, or WKT format. More specifically, if the input is a BINARY value we treat it as the WKB representation, whereas if the input is a STRING value we detect whether it is GeoJSON or WKT.	

Example(s)	`-- NOTICE: difference between `st_geogperimeter` [meters] and `st_perimeter` [units: e.g. degrees]`
	`select`
	`  st_geogperimeter(g) as dbx_perim_meters,`
	`  st_perimeter(g) as dbx_perim_units`
	`from (`
	`  select`
	`  'POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32.5700982923276, -115.428003926578 32.5701189200886, -115.428049576338 32.5700249817816, -115.427990375581 32.5700043540444))' as g`
	`  )`	

	1.2 dbx_perim_meters	1.2 dbx_perim_units
1	34.55278137088886	0.0003342688776904781

	Input type(s)	Output type
st_point	(DOUBLE, DOUBLE[, INT])	GEOMETRY
Description	The expression takes as input two double values and an optional SRID integer. It returns a two-dimensional point GEOMETRY.	
Notes	The order of double values is X (longitude), Y (latitude). The optional SRID of the point is the one provided as third argument: • If negative, SRID is set to 0. • If not provided, SRID is set to 0.	
Example(s)	`select st_astext(st_point(-77.0257, 38.9005)) as pnt`	

	ABC pnt
1	POINT(-77.0257 38.9005)

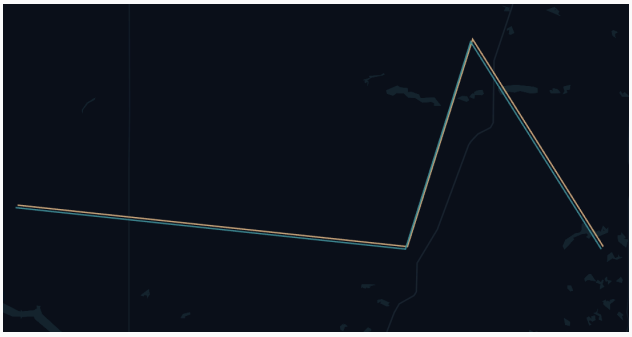
	Input type(s)	Output type
st_rotate	(GEOMETRY, DOUBLE) / (STRING, DOUBLE) / (BINARY, DOUBLE)	GEOMETRY

Description	Takes as input a geometry and a rotation angle (in radians) and rotates the input geometry by the input angle around the Z axis.
Notes	Be mindful of the variations of degrees at various latitudes and longitudes.

Example(s)	`-- Here we rotate by only 0.0000005 radians`
	`select *, st_astext(st_rotate(g, 0.0000005)) as rg from samp_wkt`
	
	Using map_render utility function in Getting Started notebook (blue is 'g' and tan is 'rg'):
	
	 

	Input type(s)	Output type
st_scale	(GEOMETRY, DOUBLE, DOUBLE[, DOUBLE]) / (STRING, DOUBLE, DOUBLE[, DOUBLE]) / (BINARY, DOUBLE, DOUBLE[, DOUBLE])	GEOMETRY
Description	Takes as input a geometry and scale factors in the X, Y, and Z directions. The expression returns the scaled geometry using the provided scaling factors.	
Notes	The Z scaling factor is optional. If not specified it defaults to `1`. If the geometry does not have any Z coordinates, the Z scaling factor is ignored.	

Example(s)	`-- Here we scale X and Y each by 1.00003 degrees (assuming 4326).`
	`select *, st_astext(st_scale(g, 1.00003, 1.00003)) as sg from samp_line`
	
	Using map_render utility function in Getting Started notebook (blue is 'g' and tan is 'sg'):	

	Input type(s)	Output type
st_simplify	(GEOMETRY, DOUBLE) / (STRING, DOUBLE) / (BINARY, DOUBLE)	GEOMETRY
Description	Expression takes as input a geometry value and a tolerance value, and simplifies the geometry.	

Notes	Uses the Douglas-Peucker algorithm. Points and multipoints remain unchanged. The expression drops the M coordinates from the input geometry if any.
	• If the input is a STRING value, it is expected to be the GeoJSON or WKT description of a geometry.
	  ○ If the input is GeoJSON, the SRID of the resulting geometry is 4326.
	  ○ If the input is WKT, the SRID of the resulting geometry is 0.
	• If the input is WKB, the SRID of the resulting geometry is 0.
	• If the input is a GEOMETRY value, the SRID value of the output geometry is the same as that of the input value.	

Example(s)	`-- tolerance set to 0.75`
	`select st_astext(`
	`  st_simplify('LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)', 0.75)`
	`) as g_simp`
	
	| | g_simp |
	| 1 | LINESTRING(-100.25 50.5,-100.3 50.05) |	

	Input type(s)	Output type
st_setsrid	(GEOMETRY, INT)	GEOMETRY
st_srid	GEOMETRY / GEOGRAPHY	INT
st_transform	(GEOMETRY, INT)/(STRING, INT)/(BINARY, INT)	GEOMETRY
Description	SRID (Spatial Reference System Identifier) is initially supported in the preview.	
Notes	• GEOGRAPHY data type only supports SRID=4326 • H3 assumes SRID=4326 (WGS84 ellipsoid) • `ST_Transform` is the main function to reproject from one SRID to another For best interoperability among various spatial data, we recommend standardizing to SRID=4326, then only specially transforming as needed for various analysis or generated data products. For v1 of the preview, except for `ST_Transform` no other function utilizes the information provided by the SRID.	

Example(s)	`-- when srid is not set`
	`select st_srid(st_geomfromtext(g)) as srid, * from samp_wkt`
	
	`-- when srid is set / inferred`
	`-- Note: GeoJSON spec requires SRID=4326 [WGS84]`
	`with samp_geojson as (`
	`  select * except (g), st_asgeojson(st_geomfromtext(g)) as g from samp_wkt`
	`)`
	`select st_srid(st_geogfromgeojson(g)) as srid, * from samp_geojson`
	
	`-- let's set srid to 4326`
	`-- we will first coerce wkt to ewkb and specify SRID`
	`with samp_ewkb as (`
	`  select * except (g), st_asewkb(st_geomfromtext(g, 4326)) as g from samp_wkt`
	`)`
	`select st_srid(st_geomfromewkb(g)) as srid, * from samp_ewkb`
	
	`-- lets transform from 4326 [WGS84] to 3857 [Web Mercator Projection]`
	`-- using the same ewkb from above`
	`with samp_ewkb as (`
	`  select * except (g), st_asewkb(st_geomfromtext(g, 4326)) as g from samp_wkt`
	`)`
	`select st_astext(st_transform(st_geomfromewkb(g), 3857)) as g_3857,`
	`st_astext(st_geomfromewkb(g)) as g, * except(g) from samp_ewkb`

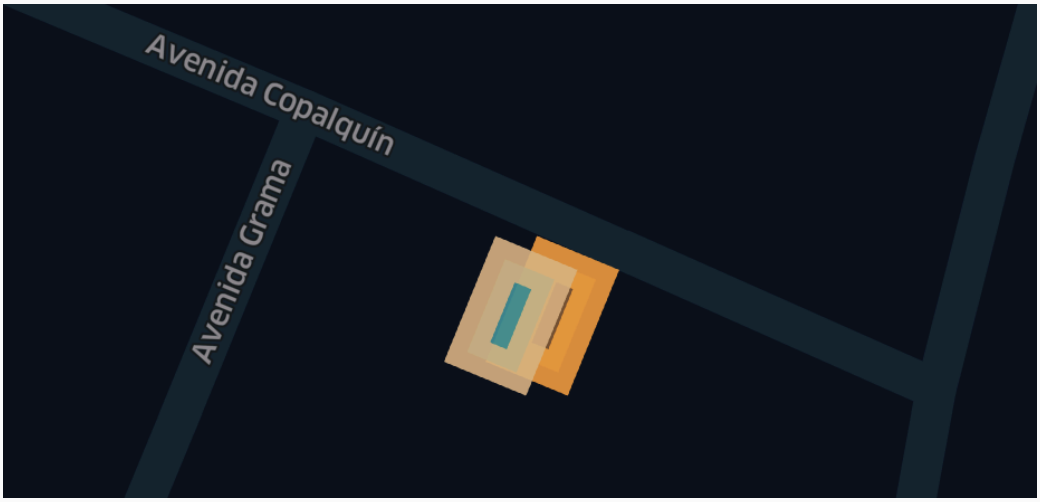
	srid	row_id	g
1	0	1	POLYGON((-115.427990375581 32.5700043540444, -115.427944725767 32...

	srid	row_id	g
1	4326	1	{"type":"Polygon","coordinates":[[[-115.427990375581,32.5700043540444],[...

	srid	row_id	1.2	g
1	4326	1	AQMAACDmEAAAAQAAAAUAAADih74xZNtcwGKZFef1SEBAj2pGcmPbXMC4hhj7+...	

	g_3857	g
1	POLYGON((-12849385.1119006 3838367.33301749,-12849380.0301865 38...	POLYGON((-115.427990375581 32.5700043540444,-115.427944725767 32...

	Input type(s)	Output type
st_translate	(GEOMETRY, DOUBLE, DOUBLE[, DOUBLE]) / (STRING, DOUBLE, DOUBLE[, DOUBLE]) / (BINARY, DOUBLE, DOUBLE[, DOUBLE])	GEOMETRY
Description	Takes as input a geometry and offsets in the X, Y, and Z directions. The expression returns the translated geometry using the provided offsets.	
Notes	The Z offset is optional. If not specified it defaults to `0`. If the geometry does not have any Z coordinates, the Z offset is ignored.	

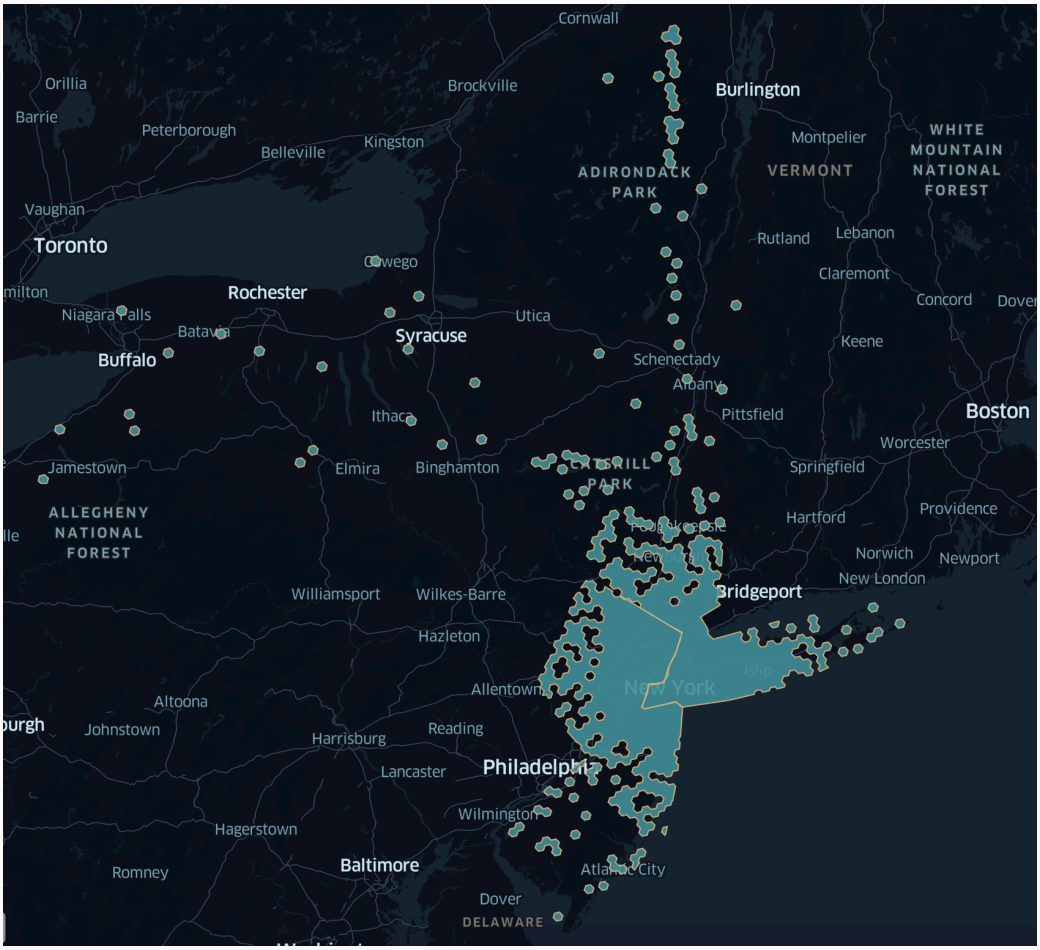
Example(s)	`-- Here we translate X (longitude) by 0.00005 degrees (assuming 4326)`
	`select *, st_astext(st_translate(g, 0.00005, 0.0)) as tg from samp_wkt`
	
	Using map_render utility function in Getting Started notebook (blue is 'g' and tan is 'sg'):
	
	 

	Input type(s)	Output type
st_union_agg	GEOMETRY / STRING / BINARY (aggregate)	GEOMETRY
Description	An aggregate expression that computes the point-wise union of all the geometries in the column as a two-dimensional geometry. It takes as input a STRING, GEOMETRY, or BINARY column. In the case of STRING column, we expected the geometries to be in WKT or GeoJSON format.	
Notes	• If the input column's type is GEOMETRY, the expression returns an error if we encounter two geometries with different SRID values. Otherwise, the SRID value of the result is equal to the common SRID value of the geometries in the input column. • If the input column's type is STRING or BINARY, the SRID value of the result will be 0, unless all input rows correspond to the same SRID value (0 for WKB and WKT, and 4326 for GeoJSON), in which case that SRID value is used for the resulting geometry.	

Example(s)	`-- example unioning h3 chips back together`
	`-- note: this will not be the entire state polygon due to`
	`-- previously performing INNER join in pickup_state_h3`
	`select state_row_id, state, st_astext(st_union_agg(chip)) as chip_union from`
	`(select distinct state_row_id, state, chip from pickup_state_h3)`
	`group by state_row_id, state`
	`order by state_row_id`	

	state_row_id	state	chip_union
1	1	New Jersey	MULTIPOLYGON(((-74.3410874289631 41.1964348476457,-74.2713661277...
2	2	New York	MULTIPOLYGON(((-73.4310103477268 40.5425754588093,-73.4628 40.536...

Using map_render utility function in Getting Started notebook:

	Input type(s)	Output type
st_x	GEOMETRY / STRING / BINARY	DOUBLE
st_xmax	GEOMETRY / STRING / BINARY	DOUBLE
st_xmin	GEOMETRY / STRING / BINARY	DOUBLE
st_y	GEOMETRY / STRING / BINARY	DOUBLE
st_ymax	GEOMETRY / STRING / BINARY	DOUBLE
st_ymin	GEOMETRY / STRING / BINARY	DOUBLE
st_zmax	GEOMETRY / STRING / BINARY	DOUBLE
st_zmin	GEOMETRY / STRING / BINARY	DOUBLE
Description	• `ST_X` and `ST_Y` take as input a point geometry, and, in the case of non-empty points, return the X and Y coordinate of the point, respectively. • `ST_XMin`, `ST_XMax`, `ST_YMin`, and `ST_YMax` take as input a geometry value. • `ST_ZMax` and `ST_ZMin`. These expressions take as input a geometry and return the maximum and minimum Z coordinate of the geometry, if such a coordinate exists.	

	In addition to accepting GEOMETRY values as input, these expressions accept STRING and BINARY values as inputs, which are expected to represent a geometry object in one of the standard geospatial formats: GeoJSON (STRING), WKB (BINARY), or WKT (STRING).
Notes	`ST_X` and `ST_Y`: For empty points, the expressions return NULL. If the input geometry is not a point an error is returned. `ST_XMin`, `ST_XMax`, `ST_YMin`, and `ST_YMax` • If the input geometry is empty (contains no non-empty points), the expressions return NULL. • If the input geometry is not empty (contains at least one non-empty point), the expressions return the minimum X, maximum X, minimum Y, and maximum Y coordinate of the points in the geometry. `ST_ZMax` and `ST_ZMin`: Z coordinates do not exist for 2D or 3DM geometries, as well as for empty geometries. In these cases the expressions return NULL.

Example(s)	`-- start from WKT point data, return x and y`
	`with samp_pnt as (`
	`  select 'POINT(-75.5 40.5)' as pnt`
	`)`
	`select st_x(pnt) as lon, st_y(pnt) as lat from samp_pnt`
	
	| | 1.2 lon | 1.2 lat |
	| 1 | -75.5 | 40.5 |

	`with samp_line1 as (`
	`  select 'LINESTRING(-100.25 50.5,-100.50 50.40,-100.25 50.35,-100.30 50.05)' as g`
	`)`
	`select`
	`  st_xmin(g) as xmin, st_ymin(g) as ymin,`
	`  st_xmax(g) as xmax, st_ymax(g) as ymax`
	`from samp_line1`
	
	| | 1.2 xmin | 1.2 ymin | 1.2 xmax | 1.2 ymax |
	| 1 | -100.5 | 50.05 | -100.25 | 50.5 |

	Input type(s)	Output type
`to_geography`	STRING / BINARY	GEOGRAPHY
`to_geometry`	STRING / BINARY	GEOMETRY
`try_to_geography`	STRING / BINARY	GEOGRAPHY
`try_to_geometry`	STRING / BINARY	GEOMETRY
Description	• `To_Geography`: Takes as input the GeoJSON, WKB, or WKT description of a geography and returns the corresponding GEOGRAPHY value. • `To_Geometry`: Takes as input the GeoJSON, WKB, or WKT description of a geometry and returns the corresponding GEOMETRY value. • `Try_To_Geography`: Takes as input the GeoJSON, WKB, or WKT description of a geography and returns the corresponding GEOGRAPHY value.	

	• `Try_To_Geometry`: Takes as input the GeoJSON, WKB, or WKT description of a geometry and returns the corresponding GEOMETRY value.
Notes	• `To_Geography` and `To_Geometry` expressions return an error in case the input is invalid. • `Try_To_Geography` and `Try_To_Geometry` expressions return NULL in case the input is invalid.
Example(s)	<no example provided>